

Second Highest Salary

```
SELECT MAX(salary) FROM Employee WHERE salary < (SELECT MAX(salary) FROM Employee);
```

Or

```
SELECT salary FROM Employee ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

Third Highest Salary

```
SELECT salary FROM Employee ORDER BY salary DESC LIMIT 1 OFFSET 2;
```

Nth Highest Salary

```
SELECT salary FROM Employee ORDER BY salary DESC LIMIT 1 OFFSET N-1;
```

Duplicate Records

```
SELECT name, COUNT(*) FROM Employee GROUP BY name HAVING COUNT(*) > 1;
```

Remove Duplicate Records

```
DELETE FROM Employee WHERE id NOT IN (SELECT MIN(id) FROM Employee GROUP BY name, salary );
```

Highest Salary in Each Department

```
SELECT department, MAX(salary) FROM Employee GROUP BY department;
```

Employees With Salary Greater Than Average Salary

```
SELECT * FROM Employee WHERE salary > (SELECT AVG(salary) FROM Employee);
```

Count Employees in Each Department

```
SELECT department, COUNT(*) FROM Employee GROUP BY department;
```

Employees Who Joined Last 30 Days

```
SELECT * FROM Employee WHERE join_date >= CURRENT_DATE - INTERVAL 30 DAY;
```

Inner Join Example

```
SELECT e.name, d.dept_name FROM Employee e INNER JOIN Department d ON e.dept_id = d.dept_id;
```

Left Join

```
SELECT e.name, d.dept_name FROM Employee e LEFT JOIN Department d ON e.dept_id = d.dept_id;
```

Find Employees Without Department

```
SELECT e.name FROM Employee e LEFT JOIN Department d ON e.dept_id = d.dept_id WHERE d.dept_id IS NULL;
```

Update Salary

```
DELETE FROM Employee WHERE salary < 30000;
```

Create Table

```
CREATE TABLE Employee (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    salary INT,  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)  
);
```

```
CREATE TABLE Department (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);
```

#fetch employee who hired after '2022-01-01'

```
select * from employees where hire_date > '2022-01-01';
```

#fetch employee who hired before '2020-01-01'

```
select * from employees where hire_date < '2020-01-01';
```

#fetch employee sort by salary ascending order

```
select * from employees order by salary asc;
```

#fetch employee sort by salary and department

```
select * from employees order by salary, department_id desc;
```

#fetch total salary per department

```
select department_id, SUM(salary) AS total_salary from employees group by department_id;
```

Create Index

```
CREATE INDEX idx_employee_name  
ON Employee(emp_name);
```

This creates an **index** on **emp_name** column.

Create Unique Index

```
CREATE UNIQUE INDEX idx_email  
ON Employee(email);
```

Unique index → No duplicate values allowed.

Create Composite Index (Multiple Columns)

```
CREATE INDEX idx_name_dept  
ON Employee(emp_name, dept_id);
```

Used when queries search using both columns:

```
SELECT * FROM Employee WHERE emp_name = 'John' AND dept_id = 2;
```

Drop Index

```
DROP INDEX idx_employee_name;
```

Check Indexes on Table

```
SHOW INDEX FROM Employee;
```

Example Table With Index

```
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    salary INT,  
    dept_id INT  
);
```

```
CREATE INDEX idx_salary  
ON Employee(salary);
```

Index is a database object that improves query performance by allowing faster data retrieval using pointers instead of full table scan.

What is SQL execution plan?

SQL execution plan shows how the database engine executes a query, including table access methods, index usage, join methods, and query cost. It helps in query performance optimization.

How to see execution plan ?

```
EXPLAIN ANALYZE SELECT * FROM Employee WHERE salary > 50000;
```